

Logging & Monitoring

To demonstrate the backend logging and monitoring system our simulated website would use, I developed a Python script that captures user activity in real time and logs key actions into structured entries. This script illustrates how system events like login attempts, file uploads, chat messages, and abuse flags are tracked with metadata such as timestamps, IP addresses, status results, and user IDs.

The system mimics realistic login behavior by accepting only predefined usernames and generates a unique IP address per session. Each action taken by a logged-in user, including successful or failed logins, uploading documents, or sending chat messages, is appended to an internal log list in dictionary format. Unsafe file uploads (e.g. .exe) are flagged and rejected, while excessive messaging is monitored and flagged as "spamming" after a threshold of five messages, demonstrating basic rate-limiting.

A dedicated administrator account (admin342) can log in to view all logs and optionally export them to a .csv file. This export includes dynamic detection of all log field names to ensure compatibility, even when log structures vary across events. This ensures the logging system remains transparent, traceable, and structured for future audit or incident response needs.

The script provides a concrete example of how Python can simulate core monitoring tasks such as content moderation flagging, behavior tracking, and automated logging workflows, which are foundational capabilities for secure digital platforms handling user-generated content.

Video demonstration:

<https://deakin.au.panopto.com/Panopto/Pages/Viewer.aspx?id=51167d2d-1613-4fd3-b3ed-b32d0058bcb4>

Programming Script Used:

```
#import the required modules
```

```
import datetime #for timestamp functionality
```

```
import random #to generate random IP addresses
```

```
import csv #for CSV export functionality
```

```
#generate a random IP address in the 192.168.1.x range
```

```

def generate_ip():

    return f"192.168.1.{random.randint(2, 254)}"

#return current UTC timestamp in ISO 8601 format

def current_timestamp():

    return datetime.datetime.now(datetime.timezone.utc).strftime("%Y-%m-%dT%H:%M:%SZ")

#list of valid usernames

valid_users = ["user_102", "user_877", "user_324", "user_3424", "admin342"]

#list to store user activity logs

activity_logs = []

#dictionary to track the number of chat messages per user (for spam detection)

chat_tracker = {}

#list of allowed file extensions for uploads

safe_extensions = [".png", ".jpg", ".jpeg", ".pdf", ".txt"]

#start the main application loop

while True:

    #prompt the user to log in

    print("(valid usernames are: user_102, user_877, user_324, user_3424, admin342):")

    username = input("Enter username: ")

    password = input("Enter password (any password will work): ")

```

```

ip = generate_ip() #generate a simulated IP address for the session

#handle invalid username attempts

if username not in valid_users:

    print("Invalid username. Please try again.")

    activity_logs.append({

        "timestamp": current_timestamp(),

        "user_id": username,

        "event": "login_attempt",

        "status": "failed",

        "reason": "Invalid username",

        "ip_address": ip

    })

    continue #restart login loop


#admin user functionality: view or export activity logs

if username == "admin342":

    view_logs = input("Would you like to see the logs? (yes/no)").lower()

    if view_logs == "yes":

        #display all stored logs

        print("\n--- Activity Logs ---")

        for log in activity_logs:

            print(log)

        print("--- End of Logs ---\n")

    #offer option to export logs to CSV

```

```

export = input("Would you like to export logs to a CSV file? (yes/no): ").lower()

if export == "yes":

    #collect all unique keys used across all log entries

    all_keys = set()

    for entry in activity_logs:

        all_keys.update(entry.keys())

    all_keys = sorted(all_keys) #sort for consistent CSV headers


    #create a filename using current timestamp

    csv_filename = f"activity_logs_{datetime.datetime.now().strftime('%Y%m%d_%H%M%S')}.csv"

    #write logs to CSV

    with open(csv_filename, mode="w", newline="") as file:

        writer = csv.DictWriter(file, fieldnames=all_keys)

        writer.writeheader()

        for entry in activity_logs:

            #ensure all keys exist in the row (fill missing with empty string)

            complete_entry = {key: entry.get(key, "") for key in all_keys}

            writer.writerow(complete_entry)

        print(f'Logs exported to {csv_filename}')

    continue #go back to login after admin actions


#log successful user login

print(f"Welcome, {username}!")

activity_logs.append({

    "timestamp": current_timestamp(),

```

```

        "user_id": username,

        "event": "login_attempt",

        "status": "success",

        "ip_address": ip
    })

#begin user session actions

while True:

    print("\nWhat would you like to do?")

    print("(1) Upload a file or document")

    print("(2) Send a chat message")

    choice = input("Enter 1 or 2: ")

    if choice == "1":

        #handle file upload

        filename = input("What is the filename?: ")

        extension = input(f"What is the extension? (include the dot, allowed: {''.join(safe_extensions)}): ")

        full_file = filename + extension

        #check file extension safety

        if extension.lower() not in safe_extensions:

            print(f"{full_file} denied due to unsafe file type.")

            activity_logs.append({

                "timestamp": current_timestamp(),

                "user_id": username,

```

```

        "event": "file_upload",

        "filename": full_file,

        "status": "denied",

        "reason": "Unsafe file type",

        "ip_address": ip
    })

else:

    print(f'{full_file} accepted and uploaded.')

    activity_logs.append({

        "timestamp": current_timestamp(),

        "user_id": username,

        "event": "file_upload",

        "filename": full_file,

        "status": "accepted",

        "ip_address": ip

    })

elif choice == "2":

    #handle chat messages with spam detection

    if username not in chat_tracker:

        chat_tracker[username] = 0 #initialize count

    print("You may send up to 5 messages. Sending too many will result in a spam flag.")

    for i in range(5):

        msg = input("Enter chat message: ")

        chat_tracker[username] += 1

```

```

        activity_logs.append({
            "timestamp": current_timestamp(),
            "user_id": username,
            "event": "chat_message",
            "message": msg,
            "status": "received",
            "ip_address": ip
        })

    #if user exceeds 5 messages, flag them for spamming and log out
    if chat_tracker[username] >= 5:
        print("Too many messages sent. You are flagged for spamming and will be logged
out.")

        activity_logs.append({
            "timestamp": current_timestamp(),
            "user_id": username,
            "event": "chat_abuse",
            "status": "spamming_detected",
            "ip_address": ip
        })

        break

    break #end session after chat

#ask user if they want to perform another action or log out
print("Type 'yes' to continue or 'no' to return to login")
back = input("Would you like to go back to options?: ").lower()

```

```
if back == "no":
```

```
    break #return to login loop
```

Screenshots:

```
python activity_logger.py X
python activity_logger.py > ...
1  #import the required modules
2  import datetime #for timestamp functionality
3  import random   #to generate random IP addresses
4  import csv      #for CSV export functionality
5
6  #generate a random IP address in the 192.168.1.x range
7  def generate_ip():
8      | return f"192.168.1.{random.randint(2, 254)}"
9
10 #return current UTC timestamp in ISO 8601 format
11 def current_timestamp():
12     | return datetime.datetime.now(datetime.timezone.utc).strftime("%Y-%m-%dT%H:%M:%SZ")
13
14 #list of valid usernames
15 valid_users = ["user_102", "user_877", "user_324", "user_3424", "admin342"]
16
17 #list to store user activity logs
18 activity_logs = []
19
20 #dictionary to track the number of chat messages per user (for spam detection)
21 chat_tracker = {}
22
23 #list of allowed file extensions for uploads
24 safe_extensions = [".png", ".jpg", ".jpeg", ".pdf", ".txt"]
25
26 #start the main application loop
27 while True:
28     #prompt the user to log in
29     print("(valid usernames are: user_102, user_877, user_324, user_3424, admin342):")
30     username = input("Enter username: ")
31     password = input("Enter password (any password will work): ")
32     ip = generate_ip() #generate a simulated IP address for the session
33
34     #handle invalid username attempts
35     if username not in valid_users:
36         print("Invalid username. Please try again.")
37         activity_logs.append({
38             "timestamp": current_timestamp(),
39             "user_id": username,
40             "event": "login_attempt",
41             "status": "failed",
42             "reason": "Invalid username",
43             "ip_address": ip
44         })
45         continue #restart login loop
46
47     #admin user functionality: view or export activity logs
48     if username == "admin342":
49         view_logs = input("Would you like to see the logs? (yes/no)").lower()
50         if view_logs == "yes":
51             #display all stored logs
52             print("\n--- Activity Logs ---")
53             for log in activity_logs:
```

```

PS C:\Users\James\Downloads\Python> & C:/Users/James/AppData/Local/Programs/Python/Python313/python
(valid usernames are: user_102, user_877, user_324, user_3424, admin342):
Enter username: user_102
Enter password (any password will work): password1
Welcome, user_102!

What would you like to do?
(1) Upload a file or document
(2) Send a chat message
Enter 1 or 2: 1
What is the filename?: document1
What is the extension? (include the dot, allowed: .png, .jpg, .jpeg, .pdf, .txt): .exe
document1.exe denied due to unsafe file type.
Type 'yes' to continue or 'no' to return to login
Would you like to go back to options?: yes

What would you like to do?
(1) Upload a file or document
(2) Send a chat message
Enter 1 or 2: 1
What is the filename?: document 1
What is the extension? (include the dot, allowed: .png, .jpg, .jpeg, .pdf, .txt): .txt
document 1.txt accepted and uploaded.
Type 'yes' to continue or 'no' to return to login
Would you like to go back to options?: yes

What would you like to do?
(1) Upload a file or document
(2) Send a chat message
Enter 1 or 2: 2
You may send up to 5 messages. Sending too many will result in a spam flag.
Enter chat message: test
Enter chat message: test
Enter chat message: test
Enter chat message: test
Enter chat message: test
Too many messages sent. You are flagged for spamming and will be logged out.
(valid usernames are: user_102, user_877, user_324, user_3424, admin342):
Enter username: user_877
Enter password (any password will work): password2
Welcome, user_877!

What would you like to do?
(1) Upload a file or document
(2) Send a chat message
Enter 1 or 2: 2
You may send up to 5 messages. Sending too many will result in a spam flag.
Enter chat message: hello
Enter chat message: hello
Enter chat message: hello
Enter chat message: hello
Enter chat message: hello
Too many messages sent. You are flagged for spamming and will be logged out.
(valid usernames are: user_102, user_877, user_324, user_3424, admin342):
Enter username: admin342
Enter password (any password will work): password3
Would you like to see the logs? (yes/no)yes

```

```
python activity_logger.py  activity_logs_20250802_151957.csv X
activity_logs_20250802_151957.csv
1 event,filename,ip_address,message,reason,status,timestamp,user_id
2 login_attempt,,192.168.1.185,,,success,2025-08-02T05:18:42Z,user_102
3 file_upload,document1.exe,192.168.1.185,,Unsafe file type,denied,2025-08-02T05:18:51Z,user_102
4 file_upload,document 1.txt,192.168.1.185,,,accepted,2025-08-02T05:19:02Z,user_102
5 chat_message,,192.168.1.185,test,,received,2025-08-02T05:19:10Z,user_102
6 chat_message,,192.168.1.185,test,,received,2025-08-02T05:19:11Z,user_102
7 chat_message,,192.168.1.185,test,,received,2025-08-02T05:19:12Z,user_102
8 chat_message,,192.168.1.185,test,,received,2025-08-02T05:19:13Z,user_102
9 chat_message,,192.168.1.185,test,,received,2025-08-02T05:19:15Z,user_102
10 chat_abuse,,192.168.1.185,,,spamming_detected,2025-08-02T05:19:15Z,user_102
11 login_attempt,,192.168.1.40,,,success,2025-08-02T05:19:24Z,user_877
12 chat_message,,192.168.1.40,hello,,received,2025-08-02T05:19:29Z,user_877
13 chat_message,,192.168.1.40,hello,,received,2025-08-02T05:19:31Z,user_877
14 chat_message,,192.168.1.40,hello,,received,2025-08-02T05:19:32Z,user_877
15 chat_message,,192.168.1.40,hello,,received,2025-08-02T05:19:34Z,user_877
16 chat_message,,192.168.1.40,hello,,received,2025-08-02T05:19:36Z,user_877
17 chat_abuse,,192.168.1.40,,,spamming_detected,2025-08-02T05:19:36Z,user_877
18
```

```
activity_logs_20250802_151957.csv - Notepad
File Edit Format View Help
event,filename,ip_address,message,reason,status,timestamp,user_id
login_attempt,,192.168.1.185,,,success,2025-08-02T05:18:42Z,user_102
file_upload,document1.exe,192.168.1.185,,Unsafe file type,denied,2025-08-02T05:18:51Z,user_102
file_upload,document 1.txt,192.168.1.185,,,accepted,2025-08-02T05:19:02Z,user_102
chat_message,,192.168.1.185,test,,received,2025-08-02T05:19:10Z,user_102
chat_message,,192.168.1.185,test,,received,2025-08-02T05:19:11Z,user_102
chat_message,,192.168.1.185,test,,received,2025-08-02T05:19:12Z,user_102
chat_message,,192.168.1.185,test,,received,2025-08-02T05:19:13Z,user_102
chat_message,,192.168.1.185,test,,received,2025-08-02T05:19:15Z,user_102
chat_abuse,,192.168.1.185,,,spamming_detected,2025-08-02T05:19:15Z,user_102
login_attempt,,192.168.1.40,,,success,2025-08-02T05:19:24Z,user_877
chat_message,,192.168.1.40,hello,,received,2025-08-02T05:19:29Z,user_877
chat_message,,192.168.1.40,hello,,received,2025-08-02T05:19:31Z,user_877
chat_message,,192.168.1.40,hello,,received,2025-08-02T05:19:32Z,user_877
chat_message,,192.168.1.40,hello,,received,2025-08-02T05:19:34Z,user_877
chat_message,,192.168.1.40,hello,,received,2025-08-02T05:19:36Z,user_877
chat_abuse,,192.168.1.40,,,spamming_detected,2025-08-02T05:19:36Z,user_877
```